UNIVERSITY OF COMPUTING

MINI PROJECT REPORT

Course: Python Programming

Project Title	Building a News Headline Scraper
Unit	Unit 1
Student Name(s)	Mireille G Davis
UID / Roll Number(s)	24BSC10044
Programme / Section	BSc (2024)
Faculty Name	Monika Mam
Submission Date	04/11/2026

Abstract

This mini project builds a news headline scraper in Python. The program reads the HTML of a news page, either downloaded from a web address or opened from a saved file, and extracts every headline and its link by using a CSS selector. It then cleans the text, removes repeated headlines, saves the result to a CSV file with a timestamp and prints a short summary. The project uses the requests and BeautifulSoup libraries together with the built-in csv, argparse and datetime modules. It is organised into small functions that handle errors such as network failures and missing files. The program was tested with eight automated tests and run on a sample news page, where it found 11 headline tags and saved 9 unique headlines. The project shows how Python can automate the collection of web data for reading, research and analysis.

Introduction

News websites publish a large number of headlines every day. Following several sources by hand means opening many pages, reading them and copying the interesting ones, which is slow and cannot easily be repeated. Web scraping is the automatic extraction of information from web pages. A headline scraper is one of the simplest and most useful examples: it turns an ordinary web page into a clean, structured list that can be saved, searched and analysed.

Python is a good language for this task because it has simple syntax and strong libraries for HTTP requests, HTML parsing and file handling. This mini project applies the basic Python concepts learned in the course, such as functions, loops, lists, tuples, strings, file handling and exception handling, in one complete working program.

Objectives

1. To write a Python program that downloads or reads a news web page.
2. To extract headlines and links from the page using BeautifulSoup and CSS selectors.
3. To clean the data by removing empty entries, extra spaces and duplicate headlines.
4. To store the output in a CSV file with a timestamp.
5. To handle errors so that the program stops with a clear message instead of crashing.
6. To test the program and to follow ethical rules for web scraping.

Tools & Technologies

Tool	Type	Use in this project
Python 3	Programming language	Main language; version 3.12 was used for testing.
requests	Library	Sends the HTTP GET request and returns the page HTML; supports timeouts and status checks.
BeautifulSoup (bs4)	Library	Parses HTML and finds elements with CSS selectors using select().
csv	Standard module	Writes the headlines to a CSV file.
argparse	Standard module	Reads the source, selector and output file name from the command line.
datetime	Standard module	Creates the scrape timestamp.
Browser Inspect tool	Browser feature	Used to find which HTML tag holds the headlines, so the right selector can be chosen.
Text editor / IDLE / VS Code	Development tool	Used to write and run the code.

Table 1: Tools and technologies used

System Design

Overall Flow

The program works as a simple pipeline, shown in Figure 1. The user gives a source (a web address or a saved HTML file). The program gets the HTML, finds the headline elements, cleans the list, saves it to a CSV file and prints a summary. If the page cannot be fetched or read, an error message is printed and the program stops.

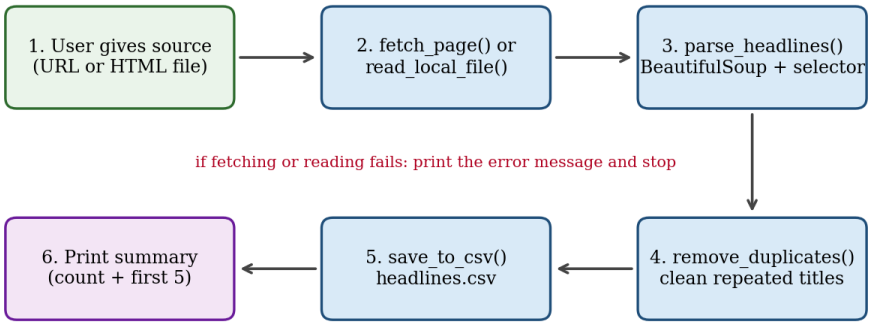


Figure 1: Overall flow of the news headline scraper

Data Set

The data set used for demonstration is a small HTML file named sample_news.html. It imitates a news home page and contains fictional headlines, so no real website or copyright content is involved and the test can be repeated without internet access. The page has 12 headline tags (h2 elements with links), including one empty headline, one headline with extra spaces and capital letters, and one exact repeat. It also contains a heading and a paragraph that are not headlines (an h1 title, an advertisement line and an h3 sports link) to check that the selector picks only the correct elements. The output data set is a CSV file with three columns: Headline, Link and Scraped at.

Main Modules

The program is divided into six functions, each with one job (Table 2).

Function	Purpose	Input / output
fetch_page(url)	Downloads a web page with a 10-second timeout and reports errors.	URL / HTML text or None
read_local_file(path)	Reads a saved HTML file; used for offline testing.	File path / HTML text or None
parse_headlines(html, selector)	Finds elements matching the selector and returns cleaned text with the link.	HTML, selector / list of (headline, link)
remove_duplicates(rows)	Removes repeated headlines, ignoring capital letters, and keeps the first one.	List / shorter list
save_to_csv(rows, filename)	Writes the rows and a timestamp to a CSV file.	List, file name / CSV file
main()	Reads command-line options, calls the other functions in order and prints a summary.	Command line / screen output

Table 2: Main modules (functions) of the program

Algorithm

The scraping algorithm has the following steps:

1. Start and read the source, the CSS selector (default "h2 a") and the output file name from the command line.
2. If the source begins with "http", download the page with fetch_page(); otherwise read the file with read_local_file().
3. If the page could not be obtained, print the error and stop.
4. Parse the HTML with BeautifulSoup and select all elements that match the selector.

5 For each element, take its text, remove extra spaces and line breaks, and skip it if it is empty. Save the text and the link in a list.

6 Go through the list; keep a headline only if its lower-case form has not been seen before.

7 Write the unique headlines, the links and the current date and time to the CSV file.

8 Print the number of headlines found, the number after removing duplicates, the file name and the first five headlines. Stop.

The time taken by the program grows in proportion to the number of headline elements (n), because each element is processed once and a set gives fast checking for duplicates. In big-O terms the cleaning and saving steps are O(n).

Implementation

The complete program, news_scraper.py (90 lines), is given in Listing 1. To use it, install the libraries with `pip install requests beautifulsoup4` and run it from the command line as shown below.

```
python news_scraper.py sample_news.html
python news_scraper.py https://example.com/news --selector "h2 a" --out news.csv
```

The selector "h2 a" means a link inside an h2 heading. Different websites use different tags, so the selector must be changed for each site after inspecting its HTML in the browser.

```
"""News Headline Scraper - Python mini project."""
import argparse
import csv
from datetime import datetime

import requests
from bs4 import BeautifulSoup

HEADERS = {"User-Agent": "Mozilla/5.0 (educational mini project)"}


def fetch_page(url, timeout=10):
    """Download a web page; return HTML text or None on failure."""
    try:
        response = requests.get(url, headers=HEADERS, timeout=timeout)
        response.raise_for_status()
        return response.text
    except requests.RequestException as error:
        print(f"Request failed: {error}")
        return None


def read_local_file(path):
    """Read an HTML file saved on the computer (used for testing)."""
    try:
        with open(path, encoding="utf-8") as file:
            return file.read()
    except OSError as error:
        print(f"Cannot read file: {error}")
        return None
def parse_headlines(html, selector="h2 a"):
    """Return a list of (headline, link) pairs matching a CSS selector."""
```

```
    soup = BeautifulSoup(html, "html.parser")
    rows = []
    for tag in soup.select(selector):
        title = " ".join(tag.get_text().split())
        if title:
            rows.append((title, tag.get("href", "")))
    return rows


def remove_duplicates(rows):
    """Drop repeated headlines (case-insensitive), keeping the first."""
    seen, unique = set(), []
    for title, link in rows:
        key = title.lower()
        if key not in seen:
            seen.add(key)
            unique.append((title, link))
    return unique


def save_to_csv(rows, filename="headlines.csv"):
    """Save headlines with a scrape timestamp to a CSV file."""
    stamp = datetime.now().strftime("%Y-%m-%d %H:%M")
    with open(filename, "w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        writer.writerow(["Headline", "Link", "Scraped at"])
        for title, link in rows:
            writer.writerow([title, link, stamp])


def main():
    parser = argparse.ArgumentParser(description="News headline scraper")
    parser.add_argument("source", help="URL (http...) or a local HTML file")
    parser.add_argument("--selector", default="h2 a", help="CSS selector")
    parser.add_argument("--out", default="headlines.csv", help="CSV file")
    args = parser.parse_args()

    if args.source.startswith("http"):
        html = fetch_page(args.source)
    else:
        html = read_local_file(args.source)
    if html is None:
        return

    found = parse_headlines(html, args.selector)
    rows = remove_duplicates(found)
    save_to_csv(rows, args.out)
    print(f"Headlines found: {len(found)}")
    print(f"After removing duplicates: {len(rows)}")
    print(f"Saved to {args.out}")
    for number, (title, _) in enumerate(rows[:5], start=1):
        print(f"{number}. {title}")


if __name__ == "__main__":
    main()
```

Listing 1: news_scraper.py

Testing & Results

Eight automated tests were written in test_news_scraper.py using assert statements, and all eight passed. The program was also run on the sample page. Table 3 lists the tests.

Test	What it checks	Result
test_parse_basic	Headlines and links are extracted and extra spaces are removed.	Pass
test_parse_skips_empty	An empty headline tag is ignored.	Pass
test_parse_empty_html	Empty HTML gives an empty list and no error.	Pass
test_other_selector	A different selector (h3 a) works.	Pass
test_duplicates	Repeats are removed regardless of capital letters; order is kept.	Pass
test_csv	The CSV file has the correct header and data.	Pass
test_missing_file	A missing file returns None with a message.	Pass
test_sample_page	The sample page gives exactly 9 unique headlines.	Pass

Table 3: Automated test cases and results

Output of the program on the sample page:

```
Headlines found: 11
After removing duplicates: 9
Saved to headlines.csv
1. City council approves new metro line
2. University announces revised exam dates
3. Heavy rain alert issued for coastal districts
4. Local start-up raises funding for solar tech
5. Hospital opens new children's wing
```

Output 1: Console output

The page contains 12 headline tags. One is empty, so 11 headlines were found. Two of these are repeats (one with different capital letters and spacing), leaving 9 unique headlines in the CSV file.

```
Headline,Link,Scraped at
City council approves new metro line,/news/metro-line,2026-11-03 12:57
University announces revised exam dates,/news/exam-dates,2026-11-03 12:57
Heavy rain alert issued for coastal districts,/news/rain-alert,2026-11-03 12:57
... (9 data rows in total)
```

Output 2: First rows of headlines.csv

Error handling was also checked. Running the program with a file that does not exist printed "Cannot read file" and stopped. Running it with an address that could not be reached (no internet connection) printed "Request failed" and stopped without a crash. The program was not run against a live news website in this project, so the selector for a real site would need to be adjusted and checked before use.

Advantages & Limitations

Advantages

- **Fast and automatic:** collects many headlines in a second, instead of copying them by hand.
- **Clean output:** a structured CSV file that opens in Excel and can be used for analysis.
- **Simple and modular:** short functions that are easy to read, test and change.
- **Reliable:** errors are handled and the code is covered by automated tests.
- **Low cost:** uses only free, open-source libraries.

Limitations

- **Depends on page layout:** if the website changes its HTML, the selector must be updated.
- **No JavaScript content:** pages that load headlines with JavaScript are not supported by requests and BeautifulSoup.
- **One page at a time:** the program does not follow links or collect several pages.
- **Limited cleaning:** only empty headlines and exact repeats (ignoring capital letters) are removed; similar headlines are kept.
- **Ethical and legal limits:** scraping must respect each website's terms of use and robots.txt file, and headlines should not be republished as the project's own content.

Future Scope

- **Several websites:** keep a selector for each site in a dictionary and scrape many sources in one run.
- **Politeness:** check robots.txt automatically and add a delay with time.sleep() between requests.
- **Database:** store headlines in SQLite and skip those already saved on earlier days.
- **Scheduling:** run the scraper automatically every hour using a scheduler.
- **Text analysis:** count common words, group headlines by topic or measure sentiment.
- **User interface:** add a small graphical or web interface to enter the URL and view results.
- **RSS and APIs:** support official feeds where available, which are more stable than scraping HTML.

Conclusion

This mini project successfully built a news headline scraper in Python. The program reads a news page from a web address or a saved file, extracts headlines and links with BeautifulSoup, cleans the data, removes duplicates and saves the result to a timestamped CSV file. It uses functions, loops, lists and tuples, string handling, file handling, exception handling and command-line arguments, and its behaviour was confirmed by eight automated tests and a run on a sample page. The project showed that a short Python program can automate a repetitive task and that good structure and error handling make a program easier to test and trust. It also showed that scraping has technical and ethical limits that must be respected.

References

[1] R. Mitchell, Web Scraping with Python: Collecting More Data from the Modern Web, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.

[2] L. Richardson, "Beautiful Soup Documentation." crummy.com/software/BeautifulSoup/bs4/doc/.

[3] Requests documentation, "Requests: HTTP for Humans." requests.readthedocs.io.

[4] Python Software Foundation, "The Python Standard Library: csv, argparse, datetime." docs.python.org.

[5] M. Koster, G. Illyes, H. Zeller and L. Sassman, RFC 9309: Robots Exclusion Protocol. Internet Engineering Task Force, 2022.

Student Signature: _____